

# Tatami

JSON Shredding and Staged Lowering to Effect-Typed OCaml

Durwasa Chakraborty · Vimala Soundarapandian · Vishakh Desai

Department of Computer Science & Engineering  
Indian Institute of Technology Madras

SegFault 2026 · IICT · Design Notes

## 1 Motivation

Querying semi-structured JSON directly is expensive: predicates over nested documents cannot exploit the physical layout or index structures that a relational engine provides. Writing against a declared SQL schema, by contrast, is cheap and well-understood.

The premise of this project is therefore to *shred* JSON documents into a relational schema, derive typed accessors from that schema, and then progressively lower those accessors into a form amenable to query optimization, including heterogeneous (CPU/GPU) execution.

The work proceeds in three phases.

## 2 Phase 1 — Schema inference and module generation

Given a corpus of JSON data, we infer a relational schema and shred the documents into it. Inference is not purely structural: it recovers and preserves the integrity constraints implied by the data.

- **Referential integrity** — nested and repeated substructures are normalized into separate relations linked by generated keys, with foreign-key constraints materialized rather than left implicit.
- **Nullability** — fields are classified as total or partial, so that non-null constraints are enforced by the schema rather than checked defensively at every use site.

From this schema we emit OCaml modules that expose the shredded relations as typed values. This phase establishes the ground truth: the generated code is the authoritative description of the data model.

### 2.1 Worked example

**Step 1: the document.** Take the smallest document that still has structure in it:

```
{ "a": 1,  
  "b": { "c": 1, "d": 2 } }
```

a is a scalar. b is not: it is a nested object.

**Step 2: split it into tables.** A scalar fits in a column. A nested object does not, so it becomes a table of its own and the parent keeps a reference to it. Two tables come out:

| Table root |   |      | Table b |   |   |
|------------|---|------|---------|---|---|
| id         | a | b_id | id      | c | d |
| 1          | 1 | 1    | 1       | 1 | 2 |

Two things are recorded here that the JSON only implied:

- `root.b_id` is a foreign key into `b.id`. Every value in that column must name a row that exists. That is the referential integrity constraint.
- `b` was present in the document, so `b_id` is declared `NOT NULL`. Had some documents omitted `b`, the column would instead be nullable and the generated OCaml would carry an `option`.

The `id` columns are ours, not the document's. Shredding has to invent keys because the parent and child now live in separate tables and need something to join on.

**Step 3: the generated OCaml.** One table becomes one module. Its rows become one record type:

```
(* generated.mli *)

module B : sig
  type id
  type t = { id : id; c : int; d : int }
  val get : id -> t
end

module Root : sig
  type id
  type t = { id : id; a : int; b : B.id }
  val get : id -> t
  val b : t -> B.t      (* follows the foreign key into table b *)
end
```

**Step 4: the correspondence.**

| Relational            | OCaml                                          |
|-----------------------|------------------------------------------------|
| table                 | module                                         |
| row                   | value of type <code>t</code>                   |
| column                | record field                                   |
| primary key           | abstract type <code>id</code>                  |
| foreign key           | field holding another module's <code>id</code> |
| joining on it         | accessor function ( <code>Root.b</code> )      |
| <code>NOT NULL</code> | plain field                                    |
| nullable              | <code>option</code> field                      |

The original document is no longer stored anywhere as a document. It is reconstructed by starting at a `Root.t` and calling accessors to walk back down into the children. Phase 2 is entirely about what those accessor calls do.

### 3 Phase 2 — Effectful lowering

Phase 1 produces conventional record-style modules with plain getters and setters. Phase 2 is a source-to-source pass over that generated code which replaces those accessors with **algebraic effects**.

The change is small and entirely mechanical. Phase 1 emits, for the field `a`:

```
val a : Root.t -> int
let a r = r.a          (* returns the field *)
```

Phase 2 rewrites it so the accessor *yields* the field rather than returning it:

```
type _ Effect.t += Yield_a : int -> unit Effect.t

let a r = Effect.perform (Yield_a r.a)    (* yields the field *)
```

Reading a field is now an operation performed against whatever handler is installed, instead of a value handed straight back to the caller. Every generated getter is rewritten this way, so the generated module becomes a set of generator objects rather than a set of plain projections.

What the handlers then do with a yielded field is a separate question, and is deliberately left open here.

### 3.1 Consuming a generator

The yielding accessor plus a handler is a generator object: something that hands back one value at a time and reports exhaustion.

```
val next : unit -> int option    (* Some v, or None when finished *)
```

A consumer that has to sum a column therefore polls until it sees **None**:

```
let sum next =
  let rec go acc =
    match next () with
    | None -> acc
    | Some x -> go (acc + x)
  in
  go 0
```

Nothing is built up to be summed. At any instant the live data is one **int** and one accumulator, and the loop is tail recursive, so the stack is constant too. The list that the same computation would have needed in Phase 1 never exists.

### 3.2 Scaling posture

This biases the design toward horizontal scaling.

A materialized **int list** is a single object with a single owner. The whole relation has to be assembled somewhere before it can be consumed, so the machine holding it has to be large enough for all of it, and the only way to handle more data is a bigger machine.

A generator has neither property. Its state is one element and an accumulator, so a consumer is cheap enough to replicate: run many of them at once, each polling its own partition, and combine the partial results at the end.

```
(* n independent pollers, one reduction at the end *)
let total = List.map sum generators |> List.fold_left ( + ) 0
```

Adding a machine adds another poller, and throughput grows with the number of pollers rather than with the memory of any one of them. The pull interface is what makes this work: consumers share no structure, so there is nothing to coordinate or keep coherent between them.

This is also what Phase 3 schedules over. A fragment that is expressed as polling is an independently dispatchable unit of work, which is why it can be placed on another core, another

node, or a GPU without changing the code that consumes it.

### 3.3 Other refinements

The same pass makes other representation changes of the same kind, for example collapsing a string-encoded enumeration into an OCaml variant. These are not described here.

## 4 Phase 3 — Query lowering and heterogeneous execution

Once the program is expressed over generated modules, the provenance of every relation is known: for each file we can determine which code is responsible for populating which part of the database. This gives the compiler a complete dataflow picture, which Phase 3 exploits.

### 4.1 Distillation

Queries are written in our OCaml-embedded query language and then *distilled*: the query is decomposed into fragments, and each fragment is annotated with the execution target it can legally be lowered to — CPU or GPU.

### 4.2 Plan enumeration and costing

Lowering is driven by a repository of join implementations and physical operators. The resulting plan is a tree whose alternatives can be enumerated and costed, so we can evaluate which formulation of a given query is expected to execute better on which device.

## 5 Evaluation strategy and goal

The end goal is differential. For each query we:

1. execute the plan **as written**;
2. execute the plan produced under the optimized lowering and device-placement strategy;
3. collate the two.

This gives both a correctness oracle (the results must agree) and a direct measurement of what the optimization pipeline buys. That comparison — naive execution against optimized, heterogeneous execution over a schema shredded and typed by Phases 1 and 2 — is the substance of the project.

## 6 Acknowledgment: the decomposable-kernel idea is BOSS’s

The idea that a query engine should be built out of *separable kernels*, each free to execute on a different device, and that a plan should be split across them at run time rather than compiled for one target, is not ours. It is the architecture of **BOSS** — Mohr-Daurat, Sun and Pirk, PVLDB 17(4):877–890, 2023 — from Holger Pirk’s group. BOSS composes Apache Arrow for storage, the GPU kernel ArrayFire, and the CPU-oriented Velox kernel into a single system via partial query evaluation over a unified exchange format for plans and data, and beats CPU-only Velox as well as the GPU-only HeavyDB on some queries. The earlier **Voodoo** — Pirk, Moll, Zaharia and Madden, PVLDB 9(14), 2016 — is the vector algebra that abstracts over cache, SIMD and core count and emits OpenCL for both CPU and GPU.

Phase 3 adopts that decomposition wholesale. What we take from BOSS is the premise that kernel boundaries are the right unit of dispatch; what we are asking on top of it is narrower, and is a compiler question rather than a systems one: *can the dispatch criterion be derived from the*

*effect signature of the generated accessors, rather than from a cost model?* We are not competing with BOSS, and we cite it as the system that does the full thing.

## 7 References

- **BOSS** — Mohr-Daurat, Sun, Pirk. *BOSS: An Architecture for Database Kernel Composition*. PVLDB 17(4):877–890, 2023.
- **Compiling PL/SQL Away** — Hirn, Grust. CIDR 2021.
- **Argo** — Chasseur, Li, Patel. *Enabling JSON Document Stores in Relational Systems*. WebDB 2013.
- **JSON schema inference** — Baazizi et al.
- **OCaml 5 effects** — Sivaramakrishnan et al. *Retrofitting Effect Handlers onto OCaml*. PLDI 2021.
- **quicktype**, Prisma introspection, Airbyte basic normalization — the tool-space prior art for Phase 1 alone.